Специальность: 5-04-0612-02 «Разработка и сопровождение программного обеспечения информационных систем»	Учебная дисциплина: «Основы кроссплатформенного программирования»

Лабораторная работа № 6
Инструкционно-технологическая карта

Тема: Разработка, отладка и испытание программ с использованием инкапсуляции. Модификаторы доступа

Цель: Научиться создавать классы с использованием инкапсуляции, в том числе с геттерами и сеттерами, научиться их различать, а также научиться работать с модификаторами доступа в языке Java.

Время выполнения: 2 часов

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Изучить теоретические сведения;
2. Ответить на контрольные вопросы;
3. Откомпилировать примеры программ из раздела «Теоретические сведения»;
4. Выполнить ИДЗ.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ДЛЯ ВЫПОЛНЕНИЯ РАБОТЫ

В программировании очень важна безопасность. В ООП безопасность обеспечивается по-своему - с помощью принципа инкапсуляции (с англ. "encapsulation"). Инкапсуляцию можно перевести как «положить что-то во что-то», или для простоты "обернуть в капсулу"

С помощью инкапсуляции мы защищаем данные от неправомерного использования.

Точно так же и в Java - мы пользуемся разными средствами для обеспечения принципа инкапсуляции. Но как же мы это делаем? Есть несколько способов регулировать доступ к нашим данным. Основные это:

- Модификаторы доступа (Access modifiers)
- Геттеры и Сеттеры (Getters and Setters)

Модификаторы доступа

Модификаторы доступа — это специальные слова, которые показывают, кому нельзя, а кому можно пользоваться данными.

Существуют четыре модификатора доступа:

- public - "публичный, доступный всем"

▪ default - "по умолчанию". Когда мы не пишем модификатора доступа (как мы это делали в наших предыдущих уроках), он по умолчанию имеет значение default. Данные с этим модификатором видны в пределах package.

▪ protected - "защищенный". На самом деле это то же самое, что и default, только доступ имеют еще и классы-наследники.

▪ private - "частный, личный". Такие данные видны только самому классу.

Модификаторы доступа пишутся перед названиями переменных, методов и даже классов:

Как это обеспечивает безопасность? Давайте попробуем создать класс, в котором будет только одна переменная - String s. Допустим она имеет модификатор public:

```
1
    class MyClass {
2
     public String s = "Hello World!"; 3
    }
```

Теперь попробуем вывести значение этой переменной на экран:

```
1
2
    public class Test {
3
     public static void main(String[] args) { 4
     MyClass obj = new MyClass(); 5
     System.out.println(obj.s);
6
     }
7
    }
8
```

Получили:

```
Hello World!

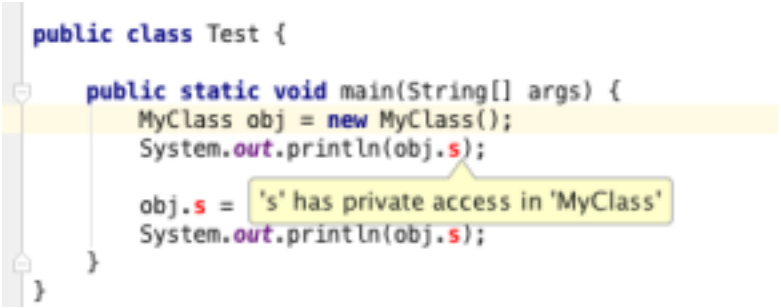
Process finished with exit code 0
```

Тем не менее, точно так же мы можем и поменять эту переменную.

Если мы не хотим этого - поменяем модификатор с public на private:

```
1
    class MyClass {
2
     private String s = "Hello World!"; 3
    }
```

Теперь при попытке доступа к переменной s - будь то чтение или запись - у нас возникнет ошибка:



Вот так мы защитили переменную.

Геттеры и Сеттеры

Ну, вот мы защитили переменную - но ее же надо как-то менять? Если влиять на переменную напрямую считается плохой практикой, то, как по другому, правильно можно изменять переменные?

Для этого существуют специальные методы - так называемые Геттеры и Сеттеры. Ну, они не то чтобы специальные - просто настолько часто используются, что были вынесены в отдельную категорию методов.

Геттер - от англ. "get", "получать" — это метод, с помощью которого мы получаем значение переменной, т.е. ее читаем. Например, создадим Геттер для нашей переменной s:

Сеттер - от англ. "set", "устанавливать" — это метод, с помощью которого мы меняем, или задаем значение переменной. Допишем Сеттер для переменной s:

```
class MyClass {
 private String s = "Hello World!";

 public String getString()
 {
 return s;
 }
 public void setS(String newValue)
 {
 s = newValue;
 }

 }
```

Теперь, если переписать наш main() по-правильному, получим:

```
1
      public class Test {
2
3
       public static void main(String[] args) { 4
       MyClass obj = new MyClass(); 5
       System.out.println(obj.getS()); 6
7
```

```
        obj.setS("It's modified!");
 8
        System.out.println(obj.getS()); 9
        }
 1
       }
```

Как видите, вместо того чтобы напрямую стучаться к переменной, мы меняем ее с помощью геттеров и сеттеров. Мы получаем:

```
Hello World!
It's modified!

Process finished with exit code 0
```


Если результат такой же, зачем это все было менять?

Тут у нас в каждом методе всего по одной строчке, но если нам понадобится добавить какую-то логику - например, присваивать новое значение строке s только если она больше какой-то длины, или если содержит слово "Java". Главное, мы получаем контроль над происходящим - никто не может просто так менять или читать наши переменные.

Доступ к элементам класса, которые объявлены с модификатором доступа private.

Элементы класса, которые объявленные с модификатором доступа private скрыты. К ним имеют доступ только методы данного класса. Из всех других частей программного кода к private-членам класса нет доступа.

Такой способ скрытия данных в классе эффективный в случаях, если нужно:

• спрятать детали организации структур данных и реализации данного класса;

• избегнуть возможные случайные изменения данных в экземпляре класса;

• обеспечить удобство доступа к данным в классе с помощью специально разработанных методов. Здесь имеется ввиду разработка методов в классе которые обеспечивают чтение/запись данных класса. Эти методы могут включать соответствующие проверки на корректность при изменении данных в классе;

• избегнуть возможные злоупотребления данными в классе что может привести к возникновению трудноуловимых ошибок в программе. Пример.

Объявляется класс, реализующий день недели. В классе объявляется скрытый (private) член данных класса с именем value. Для доступа к value в классе используются:

• конструктор DayWeek(), который инициализирует значение value=1. Конструктор объявлен без модификатора. Это значит, что в пределах пакета он имеет тип доступа public;

•метод Get(), возвращающий значение value. Поскольку метод объявлен в классе DayWeek, то в теле метода есть доступ к переменной value; •метод Set(), устанавливающий новое значение value. Метод также объявлен в классе, поэтому имеет доступ к value. В методе осуществляется проверка на

корректность значения value в границах 1..7. Если задать другое значение, то значение value не изменится;

•метод GetName(), возвращающий название дня недели в зависимости от значения value.

Реализация класса DayWeek следующая

```
// класс, который реализует день недели
class DayWeek {
 private int value; // скрытая переменная класса

 // конструктор класса, инициализирует переменную value
 DayWeek() {
```



```
 value = 1; // по умолчанию - "Monday"
 }
 // методы доступа
 // метод, который возвращает данные в классе
 public int Get() {
 return value;
 }
 // метод, который устанавливает новый день недели
 public void Set(int _value) {
 // в методе выполняется проверка на корректность значения _value  if
((_value>=1)&&(_value<=7))
 value = _value;
 }
 // дополнительный метод
 // возвращает название дня недели, которому соответствует value
 public String GetName() {
 String day = "Monday";
 switch (value) {
 case 2: day = "Tuesday"; break;
 case 3: day = "Wednesday"; break;
 case 4: day = "Thursday"; break;
 case 5: day = "Friday"; break;
 case 6: day = "Saturday"; break;
 case 7: day = "Sunday"; break;
 }
 return day;
 }
 }
```

Доступ к элементам класса, которые объявлены с модификатором доступа protected.

Рассматривают два случая доступа к protected-элементу класса: • в пределах пакета, в котором класс с данным protected-элементом объявлен (в текущем пакете);

• из другого пакета.

Если в некотором пакете элемент класса объявлен с ключевым словом protected, то:

• в текущем пакете этот элемент есть видимый из любого кода (также как и public-элемент);

• в другом пакете этот элемент есть видимым только в производных классах.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое инкапсуляция?
2. Что такое иерархия классов?
3. Как наследуются поля и методы классов?
4. Охарактеризуйте модификаторы доступа (Access modifiers)
5. Охарактеризуйте Геттеры и Сеттеры (Getters and Setters) 6. Охарактеризуйте public
7. Охарактеризуйте default

8. Охарактеризуйте protected
9. Охарактеризуйте private

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Задание 1.

1. Класс “Автомобиль” (Car): Поля: “brand”, “model”, “year”, “mileage” int). Конструктор. Геттеры и сеттеры. Метод “drive(int km)” (увеличивает пробег). Метод “displayInfo()”. В “main” создать машину, “проехать” 100 км и вывести обновленный пробег.

2. Класс “Товар” (Product): Поля: “name”, “price” (double), “quantity” (int). Конструктор. Геттеры и сеттеры. Метод “totalCost()” (возвращает price * quantity). В “main” создать товар и вывести общую стоимость.

3. Класс “Билет” (Ticket): Поля: “event”, “seatNumber”, “price”. Конструктор. Геттеры. Метод “applyDiscount()”. В “main” создать билет и применить скидку.

4. Класс “Дробь” (Fraction): Поля: “numerator”, “denominator” (int). Конструктор. Методы: “toDecimal()” (double), “toString()” (в формате “числитель/знаменатель”). Создать дроби 1/2, 3/4 и вывести их.

5. Класс “Здание” (Building): Поля: “address”, “floors” (int), “totalArea”. Конструктор. Геттеры. Метод “averageAreaPerFloor()”. В “main” создать здание и вывести среднюю площадь этажа.

6. Класс “Игра” (Game): Поля: “title”, “genre”, “rating” (int). Конструктор. Геттеры. Метод “isTop()” (true, если рейтинг >= 9). В “main” создать игру и проверить.

7. Класс “Квартира” (Apartment): Поля: “address”, “rooms” (int), “area” (double). Конструктор. Геттеры и сеттеры. Метод “pricePerSquareMeter(double totalPrice)”. В “main” создать квартиру и вывести цену за квадратный метр.

8. Класс “Билет” (Ticket): Поля: “event”, “seatNumber”, “price”. Конструктор. Геттеры. Метод “applyDiscount()”. В “main” создать билет и

применить скидку.

9. Класс “Книга” (Book): Поля: “title”, “author”, “year” (int). Конструктор. Геттеры. Метод “isOld()” (возвращает true, если книга старше 50 лет). В “main” создать 3 книги и вывести только старые.

10. Класс “Круглый стол” (RoundTable): Поля: “diameter” (double), “material”. Конструктор. Метод “area()”. Геттеры. В “main” создать стол и вывести его площадь.

11. Класс “Магазин” (Store): Поля: “name”, “address”, “employeeCount”. Конструктор. Геттеры и сеттеры. Метод “isLarge()” (true, если сотрудников > 50). В “main” создать магазин и проверить.

12. Класс “Ноутбук” (Laptop): Поля: “brand”, “ram” (int), “storage” (int), “price”. Конструктор. Геттеры и сеттеры. Метод “applyDiscount()”. В “main” создать ноутбук и применить скидку.

13. Класс “Здание” (Building): Поля: “address”, “floors” (int), “totalArea”. Конструктор. Геттеры. Метод “averageAreaPerFloor()”. В “main” создать здание и вывести среднюю площадь этажа.

14. Класс “Окружность” (Circle): Поля: “radius” (double). Конструктор. Геттеры и сеттеры. Методы: “area()”, “circumference()”. В “main” создать несколько окружностей и вывести их параметры.

15. Класс “Человек” (Person): Поля: “firstName”, “lastName”, “birthYear”. Конструктор. Геттеры. Метод “getAge()” (вычисляет возраст). В “main” создать человека и вывести его возраст.

16. Класс “Поезд” (Train): Поля: “destination”, “trainNumber”, “departureTime” (String). Конструктор. Геттеры. Метод “isDelayed(int minutes)” (изменяет время отправления). В “main” создать поезд и задержать его на 30 минут.

17. Класс “Прямоугольник” (Rectangle): Поля: “width”, “height” (double). Конструктор. Геттеры и сеттеры. Методы: “area()”, “perimeter()”. В “main” создать несколько прямоугольников и вывести их параметры.

18. Класс “Ресторан” (Restaurant): Поля: “name”, “cuisine”, “rating” (int). Конструктор. Геттеры. Метод “isPopular()” (true, если рейтинг > 4). В “main” создать ресторан и проверить.

19. Класс “Смартфон” (Smartphone): Поля: “brand”, “screenSize” (double), “batteryLife” (int). Конструктор. Геттеры и сеттеры. Метод “isGoodBattery()” (true, если > 10 часов). В “main” создать смартфон и проверить.

20. Класс “Собака” (Dog): Поля: “name”, “breed”, “age” (int). Конструктор. Геттеры и сеттеры. Метод “bark()” (выводит “Гав!”). В “main” создать собаку и заставить ее лаять.

21. Класс “Поезд” (Train): Поля: “destination”, “trainNumber”, “departureTime” (String). Конструктор. Геттеры. Метод “isDelayed(int minutes)” (изменяет время отправления). В “main” создать поезд и задержать его на 30 минут.

22. Класс “Сотрудник” (Employee): Поля: “name”, “position”, “salary” (double). Конструктор. Геттеры и сеттеры. Метод “raiseSalary(double percent)”. В “main” создать сотрудника, повысить зарплату на 10% и вывести новую зарплату.

23. Класс “Студент” (Student): Поля: “name” (String), “age” (int), “group” (String). Конструктор (все поля). Геттеры и сеттеры. Метод “displayInfo()”. В “main” создать 2 объекта и вывести их данные.

24. Класс “Счетчик” (Counter): Поля: “count” (int). Конструктор (устанавливает 0). Методы: “increment()”, “decrement()”, “getValue()”. В “main” создать счетчик, увеличить его 5 раз, уменьшить 2 раза, вывести значение.

25. Класс “Телефон” (Phone): Поля: “brand”, “model”, “price” (double). Конструктор. Геттеры. Метод “applyDiscount(double discount)” (уменьшает цену). В “main” создать телефон, применить скидку 15%, вывести новую цену.

26. Класс “Товар” (Product): Поля: “name”, “price” (double), “quantity” (int). Конструктор. Геттеры и сеттеры. Метод “totalCost()” (возвращает price * quantity). В “main” создать товар и вывести общую стоимость.

27. Класс “Точка” (Point): Поля: “x”, “y” (double). Конструктор. Геттеры. Метод “distanceTo(Point other)” (расстояние до другой точки). В “main” создать две точки и вывести расстояние.

28. Класс “Треугольник” (Triangle): Поля: “sideA”, “sideB”, “sideC” (double). Конструктор. Метод “isValid()” (проверяет существование треугольника). Метод “perimeter()”. В “main” создать треугольник и вывести периметр или сообщение об ошибке.

29. Класс “Фильм” (Movie): Поля: “title”, “director”, “year”, “rating”. Конструктор. Геттеры. Метод “isExcellent()” (true, если рейтинг >= 8). В “main” создать фильм и проверить.

30. Класс “Человек” (Person): Поля: “firstName”, “lastName”, “birthYear”. Конструктор. Геттеры. Метод “getAge()” (вычисляет возраст). В “main” создать человека и вывести его возраст.

ДОМАШНЕЕ ЗАДАНИЕ

Индивидуальное задание

ЛИТЕРАТУРА

Блинов, И. Н., Романчик, В. С. Java from EPAM : учеб.-метод. пособие / И. Н. Блинов, В. С. Романчик. Минск: Четыре четверти, 2021. 560 с.

Преподаватель А.С.Кибисова